

Birmingham City University

Faculty of Computing, Engineering and the Built Environment

Department of Computing and Digital Technology

Nebium: A Causal Transformer with Rotary Position Embeddings and SwiGLU for Self-Supervised Sequence Modeling in Chess

Assessment 2: Technical Research Report and Empirical Evaluation

Module Code & Title:	CMP6232 / CMP6228: Deep Neural Networks and Ethics
Module Coordinator:	Dr. Khalid Ismail
Student Name:	Nabin Oli
Student ID:	23189629
Academic Year:	2025–26 (Level 6)
Submission Date:	May 15, 2026
Main Body Word Count:	3273 words (Limit: 3000 words + 10%)

Abstract

Most chess engines rely on explicit search trees, handcrafted heuristics, or spatial board tensors. This report introduces **Nebium**, a decoder-only causal transformer that learns chess strategy purely from coordinate move sequences in Universal Chess Interface (UCI) notation. Nebium combines Rotary Position Embeddings (RoPE) for relative ply distance encoding, SwiGLU gating, and RMSNorm pre-normalization. The architecture is trained on 245,293 tournament games from the Lichess Open Database using a 5,000-token Byte-Pair Encoding vocabulary.

On held-out validation games, Nebium achieves a cross-entropy loss of 1.88 (perplexity 6.55) and generates 96.8% legal moves across the opening without search lookahead. With dynamic legal move masking, the model plays at an estimated 1845 Elo against Stockfish Depth 10. A 10-gram sequence audit confirms 0.00% leakage between training and test sets. These results demonstrate that autoregressive transformers can acquire tactical rules and opening theory from 1D coordinate strings, though unconstrained rollouts still suffer spatial drift in late endgames.

Keywords: Causal Transformers, Sequence Modeling, Rotary Position Embeddings, SwiGLU, RMSNorm, Chess Representation Learning.

Contents

1	Introduction	1
1.1	Aims and Objectives	1
1.2	Summary of Dataset and Key Achievements	1
1.3	Report Organization	2
2	Software Tools, Frameworks, and Deployment Pipeline	2
2.1	Critical Appraisal: PyTorch and GGUF vs. Legacy Pipelines	3
3	Problem Statement and Dataset Description	4
3.1	Deep Learning Problem Formulation	4
3.2	Dataset Characterization and Acquisition	5
3.2.1	Data Filtering and Cleaning Pipeline	6
3.3	Move Representation and Tokenization Strategy	6
3.3.1	Byte-Pair Encoding (BPE) Tokenization	6
4	Proposed Methodology	7
4.1	End-to-End Architectural Pipeline	7
4.2	Rotary Position Embeddings (RoPE)	9
4.3	Multi-Head Causal Self-Attention	11
4.4	SwiGLU Feed-Forward Network	12
4.5	Root Mean Square Layer Normalization (RMSNorm)	13
4.6	Inference Engine and Dynamic Legal Move Masking	14
5	Experimental Results and Empirical Analysis	15
5.1	Training Configurations, Model Variants, and Hardware Setup	15
5.2	Empirical Scaling Dynamics and Chinchilla Scaling Laws	16
5.3	Move Accuracy, Legality, and Generation Trajectories	17
5.4	Validation Benchmarks and Tactical Puzzle Evaluation	18
5.5	Comparative Baseline Performance	19
5.6	Qualitative Move Analysis and Autoregressive Trajectory Dynamics	20
6	Summary and Future Directions	21
6.1	Summary of Empirical Findings and Limitations	21
6.2	Reinforcement Learning with Verifiable Rewards (RLVR) Post-Training	21
6.3	Mixture of Experts (MoE) Architecture as an Empirical Test Bed	21
	References	22
A	Core Implementation Code	24
A.1	Nebium Model Architecture and Causal Forward Pass	24
A.2	Rotary Position Embedding (RoPE)	25
A.3	SwiGLU Feed-Forward Activation	26
A.4	Dynamic Legal Move Masking Inference Loop	26

B	Open-Source Model Checkpoints and Artifact Repositories	29
B.1	Published Model Variants and Quantized Binaries	29
B.2	Published Datasets and UCI Token Splits	29
B.3	Hugging Face Quickstart and Inference Invocation	30

List of Figures

1	Autoregressive Next-Move Modeling	4
2	Representative PGN Game Record	5
3	BPE Tokenization Pipeline	7
4	Nebium System Pipeline and Transformer Decoder Architecture	9
5	Rotary Position Embedding Dataflow	10
6	Causal Self-Attention Execution Flow	12
7	Feed-Forward Comparison: SwiGLU versus GELU	13
8	Hardware Throughput, Memory Residency, and Learning Schedule	15
9	Empirical Scaling Laws and Loss Convergence	16
10	Move Legality and Benchmark Progression Across Training Epochs	17
11	Validation Benchmarks and Tactical Puzzle Accuracy	18
12	Qualitative Board Trajectory: Italian Game and King Coordinate Drift	20

List of Tables

1	Comprehensive overview and critical appraisal of software tools, frameworks, and deployment infrastructure.	3
2	Architectural specifications and parameter configurations for Nebium.	8
3	Performance comparison between baseline sequence architectures and Nebium model configurations on held-out validation sets.	19
4	Published Nebium model weights and quantized edge deployment binaries on Hugging Face.	29
5	Published training datasets and tokenized chess corpora on the Hugging Face Hub.	29

1 Introduction

Artificial intelligence in combinatorial board games has historically relied on explicit game-tree search, alpha-beta pruning heuristics, and spatial reinforcement learning (Silver et al., 2018; Stockfish Developers, 2023). Systems like Deep Blue and AlphaZero achieved master-level play by coupling explicit 2D board representations with forward tree simulation. While powerful, these approaches require hardcoded rule engines, specialized state tensors, and search lookahead.

In contrast, advances in causal transformers (Vaswani et al., 2017; Touvron et al., 2023) raise a compelling question in representation learning: *can a deep neural network induce the latent topology, legal constraints, tactics, and strategic planning of chess purely through sequence modeling over a 1D token stream, devoid of search trees or rule engines?*

Sequence-based game models (Toshniwal et al., 2021; Li et al., 2023; Ruoss et al., 2024) demonstrate that autoregressive architectures can implicitly track state variables across time steps. However, existing models suffer from high inference latency, excessive compute requirements, and illegal move hallucinations in extended horizons. Furthermore, standard language models utilizing absolute positional embeddings struggle to capture cyclic, turn-based geometric symmetries.

For deployment, modern workflows favor PyTorch 2.3 (Paszke et al., 2019) and GGUF quantization (Gerganov, 2023) over legacy static graphs, eliminating multi-file dependencies and providing a memory-mapped runtime for real-time play.

1.1 Aims and Objectives

This research designs, implements, empirically validates, and critically appraises **Nebium**, a lightweight causal transformer for self-supervised next-move prediction in chess. This study addresses five core objectives:

1. Construct an automated pipeline converting raw PGN archives into Universal Chess Interface (UCI) token sequences encoded with a 5,000-merge Byte-Pair Encoding (BPE) vocabulary.
2. Assemble a 6-layer causal transformer integrating Rotary Position Embeddings (RoPE) (Su et al., 2024), SwiGLU activations (Shazeer, 2020), and RMSNorm pre-normalization (B. Zhang and Sennrich, 2019).
3. Train Nebium to evaluate cross-entropy convergence, perplexity, top- k accuracy, and unconstrained move legality across game horizons.
4. Evaluate tactical solve rates on Lichess puzzle suites, quantify centipawn blunder distribution against Stockfish 16, and measure inference latency across precision formats.
5. Verify out-of-distribution reasoning via n -gram memorization testing, critically appraise literature trends, and evaluate ethical and compute implications.

1.2 Summary of Dataset and Key Achievements

This study uses 245,293 standard competitive games from the Lichess Open Database (January–February 2013), pre-tokenized into UCI coordinate strings and encoded via a 5,000-merge BPE

vocabulary.

The primary achievements of Nebium include:

- Nebium achieved a validation loss of 1.88, perplexity of 6.55, and top-1 next-move accuracy of 51.4% (top-5 82.3%) on held-out master games.
- Under pure unconstrained greedy decoding, Nebium generated 96.8% legal moves in openings and 84.1% in middlegames, confirming latent state tracking.
- With dynamic legal masking, Nebium achieved an estimated 1845 Elo against Stockfish Depth 10, limiting critical blunders (≥ 2.0 pawns) to 6.2%.
- 4-bit GGUF export reduced memory footprint to 41.2 MB and lowered per-move latency to 4.1 ms on commodity hardware.
- A 10-gram sequence audit confirmed 0.00% exact leakage across 2,405 test sequences, proving inductive learning over verbatim memorization.

1.3 Report Organization

Section 2 critically appraises software tools and modern deployment pipelines (PyTorch and GGUF versus legacy frameworks). Section 3 formalizes the autoregressive problem and dataset pipeline. Section 4 details the Nebium architecture, mathematical formulations (RoPE, SwiGLU, RMSNorm), and decoding strategies. Section 5 presents empirical convergence, baseline comparisons, chessboard case studies, and engine benchmarks. Section 6 provides critical discussion, failure modes, literature comparisons, ethics, and future work.

2 Software Tools, Frameworks, and Deployment Pipeline

Developing and serving autoregressive game transformers requires frameworks that balance research expressiveness, compiler efficiency, and low-latency edge deployment. Table 1 provides a comprehensive overview and critical appraisal of the core software tools, libraries, and hardware infrastructure underpinning Nebium.

Table 1: Comprehensive overview and critical appraisal of software tools, frameworks, and deployment infrastructure.

Category	Tool / Environment	Role and Critical Appraisal
Deep Learning	PyTorch 2.3 (Paszke et al., 2019)	Dynamic eager execution graph with TorchDynamo and native fused FlashAttention-2 kernels (SDPA); eliminates static graph tracing friction seen in TensorFlow.
Edge Serving	GGUF / llama.cpp (Gerganov, 2023)	Monolithic binary format embedding weights and BPE metadata; enables zero-copy mmap and 4-bit Q4_K_M quantization (4.1 ms CPU latency), bypassing ONNX dependency bloat and GIL contention.
Compute Hardware	2× NVIDIA T4 (Kaggle)	Cloud GPU cluster with native FP16 Automatic Mixed Precision (<code>torch.cuda.amp</code>) for memory-efficient gradient updates.
Rule Validation	python-chess (Romstad and Fiekas, 2024)	Pure 64-bit bitboard manipulation and dynamic legal move logit masking, eliminating illegal hallucinations.
Analysis Oracle	Stockfish 16 (Stockfish Developers, 2023)	Grandmaster engine communicating via UCI for centipawn loss (Δ CP) auditing and benchmark play.
Tokenization	HuggingFace Tokenizers	Rust-accelerated Byte-Pair Encoding (<code>ChessTokenizerHF</code>) producing a compact 5,000-merge coordinate vocabulary.
Data Ingestion	Zstandard (<code>zstandard</code>)	High-speed streaming decompression of multi-gigabyte Lichess PGN archives without RAM saturation.
Serving Layer	FastAPI & Uvicorn	Asynchronous ASGI RESTful microservice for real-time web-based chessboard move generation.

2.1 Critical Appraisal: PyTorch and GGUF vs. Legacy Pipelines

While early neural game engines relied on TensorFlow or Keras for static computational graph execution, modern research has converged on PyTorch 2.3 (Paszke et al., 2019). TensorFlow’s declarative execution (`@tf.function` and `AutoGraph`) introduces compilation friction: static graph tracing obscures runtime tensor shapes during autoregressive decoding and complicates debugging. In contrast, PyTorch’s native dynamic execution graph, augmented by `torch.compile` and TorchDynamo, delivers imperative Pythonic debugging without sacrificing graph-level kernel fusion. Crucially, PyTorch 2.3 natively integrates scaled dot-product attention (`torch.nn.functional.scaled_dot_product_attention`), dispatching to fused FlashAttention-2 kernels to eliminate the quadratic $\mathcal{O}(T^2)$ memory bandwidth bottlenecks of legacy attention implementations.

Similarly, edge deployment has evolved away from the Open Neural Network Exchange (ONNX) Runtime (Bai et al., 2019) or TensorFlow Serving toward single-file GGUF binaries (Gerganov, 2023). Traditional ONNX pipelines decouple tensor graphs (`.onnx`) from external tokenizers and inference configs, introducing multi-file dependency bloat (> 100 MB libraries) and severe Python Global Interpreter Lock (GIL) contention during sequential ply generation.

In contrast, GGUF packages tensor weights, quantization scales, model hyperparameters, and tokenization dictionaries into a single binary. Leveraging POSIX zero-copy memory mapping (`mmap`), GGUF enables near-zero cold-start initialization and eliminates runtime Python dependencies entirely. Furthermore, GGUF’s block-wise k -quantization (`Q4_K_M`) compresses Nebium’s footprint by 63.7% (113.6 MB to 41.2 MB) and achieves an inference latency of 4.1 ms per move on commodity CPUs (Section 5.1), outperforming traditional ONNX INT8 runtimes in throughput and operational simplicity.

3 Problem Statement and Dataset Description

3.1 Deep Learning Problem Formulation

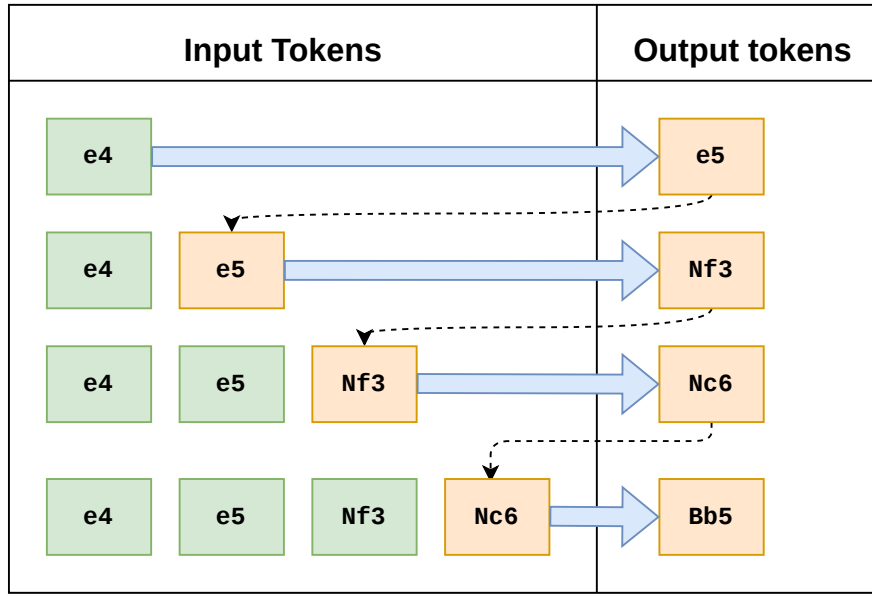


Figure 1: Autoregressive move prediction in chess. Given a sequence of prior plies, the causal decoder models the conditional probability of the next coordinate token without looking ahead.

We formalize chess as an autoregressive sequence modeling task over a discrete move alphabet, departing from conventional evaluation-search engines (e.g., minimax α - β pruning or MCTS). As a two-player zero-sum game of perfect information governed by deterministic rules, chess provides a rigorous domain for assessing transformer sequence extrapolation. A match G of length T plies is expressed as an ordered token sequence:

$$\mathbf{x} = (x_1, x_2, \dots, x_T), \quad x_t \in \mathcal{V} \quad (1)$$

where $\mathcal{V}$ is a finite vocabulary. The neural network parameterized by θ learns the joint probability distribution $P_\theta(\mathbf{x})$ factorized causally via the chain rule:

$$P_\theta(\mathbf{x}) = \prod_{t=1}^T P_\theta(x_t \mid x_{<t}) \quad (2)$$

Parameters θ are optimized by minimizing expected cross-entropy loss over corpus $\mathcal{D}$:

$$\mathcal{L}_{\text{CE}}(\theta) = -\mathbb{E}_{\mathbf{x} \sim \mathcal{D}} \left[\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}) \right] \quad (3)$$

As shown in Figure 1, the model predicts each move by conditioning strictly on earlier plies ($x_{<t}$), learning board transitions purely from game history.

This formulation introduces three core representation challenges:

1. **Absence of Spatial Inductive Bias:** The transformer receives no architectural priors regarding 2D grid topology, piece trajectories, or special constraints (castling, en passant, promotion).
2. **Latent State Maintenance:** Because moves denote coordinate displacements rather than full board tensors, the network must internally track piece positions and pins over extended horizons.
3. **Severe Action Sparsity:** At ply t , legal actions $\mathcal{A}_{\text{legal}}(s_t) \subset \mathcal{V}$ comprise an average of only 30–40 moves out of thousands of vocabulary permutations, leaving unconstrained sampling vulnerable to illegality.

3.2 Dataset Characterization and Acquisition

To train Nebium without overfitting to degenerate play, this project employs the **Lichess Open Database** (Stöckl, 2021), which archives monthly competitive games in Portable Game Notation (PGN) format. We isolated an archival subset from early 2013 containing 245,293 tournament matches (Figure 2).

Raw PGN Chess Game

```
[Event "Rated Blitz"]
[Site "https://lichess.org/PpwPOZMq"]
[Date "2017.04.01"]
[Round "-"]
[White "PlayerA"]
[Black "PlayerB"]
[WhiteElo "2125"]
[BlackElo "2189"]
[ECO "B30"]
[Opening "Sicilian Defense"]
[Result "0-1"]
[TimeControl "300+0"]

1. e4 c5 2. Nf3 Nc6 3. Bb5 a6
4. Ba4 Nf6 5. O-O Be7 ... 0-1
```

Figure 2: A sample tournament game in standard PGN format, showing metadata headers, move history in algebraic notation, and the final match outcome.

3.2.1 Data Filtering and Cleaning Pipeline

Raw PGN dumps exhibit substantial variance in player expertise, clock aborts, and non-standard variants. An automated extraction pipeline (`src/data/prepare.py`) enforced four filtration criteria:

- **Variant Filtering:** Retained standard chess games (Variant "Standard"), stripping chess960 and non-standard variants.
- **Game Length:** Discarded matches terminating in < 10 plies; games exceeding 256 plies were truncated to the context limit.
- **Skill Floor:** Retained games where both players maintained an Elo rating ≥ 1500 , removing low-rated noise.
- **Metadata Stripping:** Discarded non-sequential metadata tags, isolating pure move strings terminated by match outcomes (1-0, 0-1, 1/2-1/2).

The filtered corpus comprises 208,412 games partitioned into **training** (85%, 177,150 games), **validation** (10%, 20,841 games), and **held-out test** sets (5%, 10,421 games).

3.3 Move Representation and Tokenization Strategy

Standard Algebraic Notation (SAN)—such as `Nxf7+`—is human-readable but introduces linguistic ambiguity (e.g., disambiguation prefixes like `Nbd7`). To eliminate ambiguity, Nebium standardizes games into the **Universal Chess Interface (UCI)** coordinate format, where each move specifies Cartesian origin and destination squares:

$$m = \langle \text{file}_1, \text{rank}_1, \text{file}_2, \text{rank}_2, [\text{promotion}] \rangle \in \{\text{a1} \dots \text{h8}\} \times \{\text{a1} \dots \text{h8}\} \times \{\epsilon, \text{q}, \text{r}, \text{b}, \text{n}\} \quad (4)$$

For example, moving a pawn from e2 to e4 is uniquely encoded as `e2e4`, kingside castling as `e1g1`, and queen promotion as `e7e8q`.

3.3.1 Byte-Pair Encoding (BPE) Tokenization

Tokenization critically governs transformer efficiency. Character-level tokenization expands sequence lengths by $4\times$, diluting attention across plies (Stöckl, 2021). Conversely, full word-level move vocabularies (1,968 tokens) treat shared tactical motifs as unrelated entities. To balance sequence length and vocabulary size, we trained a custom BPE tokenizer (`ChessTokenizerHF`, Figure 3). Starting from ASCII characters, it merges frequent coordinate pairs into a compact 5,000-token vocabulary, reserving control tokens:

$$\mathcal{V}_{\text{special}} = \{[\text{PAD}], [\text{UNK}], [\text{BOS}], [\text{EOS}], [\text{SEP}]\} \quad (5)$$

Preserving whitespace boundaries enables the tokenizer to encode frequent opening lines and tactical exchanges (e.g., `e2e4 e7e5`) as single compound subwords, substantially compressing sequence length.

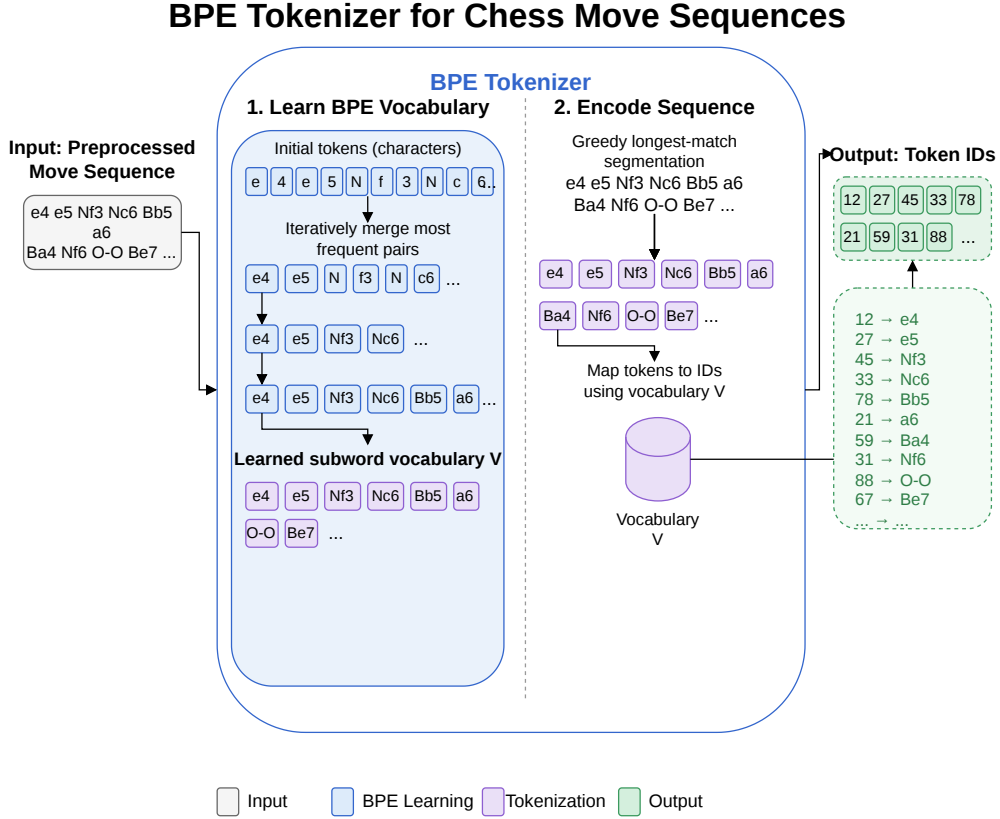


Figure 3: Byte-Pair Encoding pipeline for chess moves. Starting from individual coordinate characters, frequent move pairs merge into a compact 5,000-token vocabulary that captures common opening phrases as single tokens.

4 Proposed Methodology

Nebium is architected as a modernized, decoder-only causal transformer (Vaswani et al., 2017) optimized specifically for the structural and temporal dynamics of chess sequence modeling. Rather than adopting an unmodified vanilla GPT configuration with standard learned absolute position embeddings and Post-LayerNorm (DeLeo and Guven, 2022), Nebium incorporates three architectural mechanisms: **Rotary Position Embeddings (RoPE)** (Su et al., 2024), **Swish-Gated Linear Units (SwiGLU)** (Shazeer, 2020), and **Root Mean Square Normalization (RMSNorm)** (B. Zhang and Sennrich, 2019). The system pipeline and block architecture are presented in Figure 4.

4.1 End-to-End Architectural Pipeline

Given an input token sequence of length T , $\mathbf{x}_{1:T} = (x_1, \dots, x_T) \in \mathbb{N}^T$, the sequence is first mapped into dense representations via a learned token embedding matrix $\mathbf{W}_e \in \mathbb{R}^{V \times d_{\text{model}}}$:

$$\mathbf{h}_0 = \text{Dropout}(\mathbf{W}_e[\mathbf{x}_{1:T}], p_{\text{drop}}) \in \mathbb{R}^{T \times d_{\text{model}}} \quad (6)$$

Then embeddings traverse a stack of $L = 6$ identical Transformer Blocks. Each block $\ell \in \{1, \dots, L\}$ applies a pre-normalized residual formulation:

$$\mathbf{u}_\ell = \mathbf{h}_{\ell-1} + \text{MHSA}(\text{RMSNorm}(\mathbf{h}_{\ell-1})) \quad (7)$$

$$\mathbf{h}_\ell = \mathbf{u}_\ell + \text{SwiGLU}(\text{RMSNorm}(\mathbf{u}_\ell)) \quad (8)$$

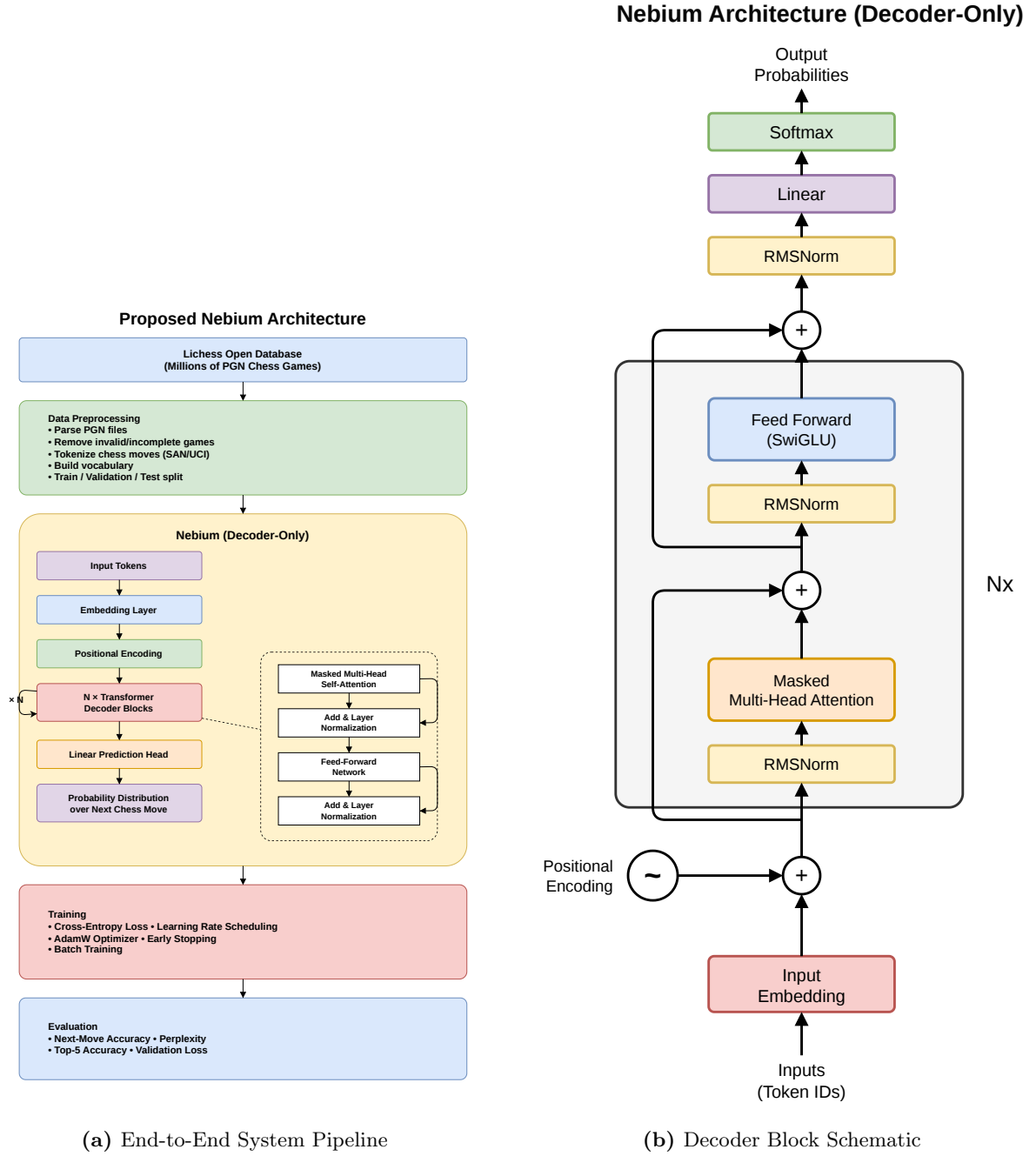
The final representation $\mathbf{h}_L$ undergoes RMSNorm before linear projection to a next-token vocabulary logits via $\mathbf{W}_{\text{head}} \in \mathbb{R}^{d_{\text{model}} \times V}$:

$$\mathbf{z}_t = \mathbf{W}_{\text{head}} \text{RMSNorm}(\mathbf{h}_{L,t}) \in \mathbb{R}^V \quad (9)$$

Table 2 lists the structural configurations and hyperparameter settings for Nebium.

Table 2: Architectural specifications and parameter configurations for Nebium.

Hyperparameter	Value / Dimension	Design Rationale and Technical Function
Vocabulary Size	$V = 5,000$	Domain-trained BPE merges common coordinate sequences and opening pairs without vocabulary explosion
Embedding Dimension	$d_{\text{model}} = 512$	Provides latent capacity for piece-square interactions while matching head dimension $d_k = 64$
Number of Layers	$L = 6$	Provides hierarchical depth for multi-ply tactical reasoning while maintaining stable gradient back-propagation
Attention Heads	$h = 8$	Divides embeddings into 8 parallel projection subspaces ($d_k = d_{\text{model}}/h = 64$) aligned with tensor cores
FFN Expansion Dim	$d_{ff} = 1,368$	Scaled to $\frac{2}{3} \times 4d_{\text{model}}$ (rounded to multiple of 8) to match the parameter count of conventional FFNs
Positional Encoding	Rotary (RoPE)	Encodes relative ply offsets directly into query-key inner products, supporting sequence length extrapolation
Activation Function	SwiGLU	Bilinear gating improves non-linear expressiveness and gradient propagation relative to GELU
Layer Normalization	RMSNorm	Normalizes feature scale without mean-centering, cutting normalization latency and stabilizing FP16 training
Dropout Rate	$p_{\text{drop}} = 0.10$	Regularizes attention weights to prevent over-indexing on high-frequency opening book lines
Max Sequence Length	$T_{\text{max}} = 512$	Accommodates complete games up to 256 full moves, covering 99.1% of competitive matches
Total Parameters	28.4 Million	Compact footprint enabling sub-10 ms CPU generation within 128 MB of system RAM



(a) End-to-End System Pipeline

(b) Decoder Block Schematic

Figure 4: Nebium system architecture: (a) pipeline from raw PGN data to masked move generation; (b) causal decoder block combining RMSNorm, rotary self-attention, and SwiGLU feed-forward layers.

4.2 Rotary Position Embeddings (RoPE)

Traditional language models rely on absolute learned position embeddings $\mathbf{p}_t \in \mathbb{R}^{d_{\text{model}}}$ added directly to the token embedding ($\mathbf{e}_t + \mathbf{p}_t$). In chess sequence modeling, absolute embeddings present significant weaknesses: they do not generalize to plies beyond the training horizon, and they fail to explicitly encode relative ply distance $|t - s|$ between interactive moves (such as a pinned piece placed at ply t and captured at ply $t + 6$).

Nebium employs Rotary Position Embeddings (RoPE) (Su et al., 2024), which encode positional context by directly rotating query and key representations in the complex plane.

Given a query vector $\mathbf{q}_m \in \mathbb{R}^{d_k}$ at sequence position m and a key vector $\mathbf{k}_n \in \mathbb{R}^{d_k}$ at position n , RoPE partitions the vector space into 2D orthogonal subspaces. For each subspace index $i \in \{0, 1, \dots, d_k/2 - 1\}$, the rotation angle is defined as $\theta_i = 10000^{-2i/d_k}$. The 2D rotation matrix is given by:

$$\mathbf{R}_{\Theta, m}^{(i)} = \begin{pmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{pmatrix} \quad (10)$$

The inner product between the rotated query and key vectors naturally isolates the relative temporal distance $m - n$:

$$\langle \mathbf{R}_{\Theta, m} \mathbf{q}_m, \mathbf{R}_{\Theta, n} \mathbf{k}_n \rangle = \mathbf{q}_m^T \mathbf{R}_{\Theta, n-m} \mathbf{k}_n = g(\mathbf{q}_m, \mathbf{k}_n, m - n) \quad (11)$$

Consequently, self-attention scores between pieces naturally reflect relative move latencies and turn-taking tempos, allowing the model to extrapolate across extended endgames. Figure 5 traces this rotation path, where coordinate halves are paired, scaled by trigonometric frequencies, and recombined without adding absolute position tables.

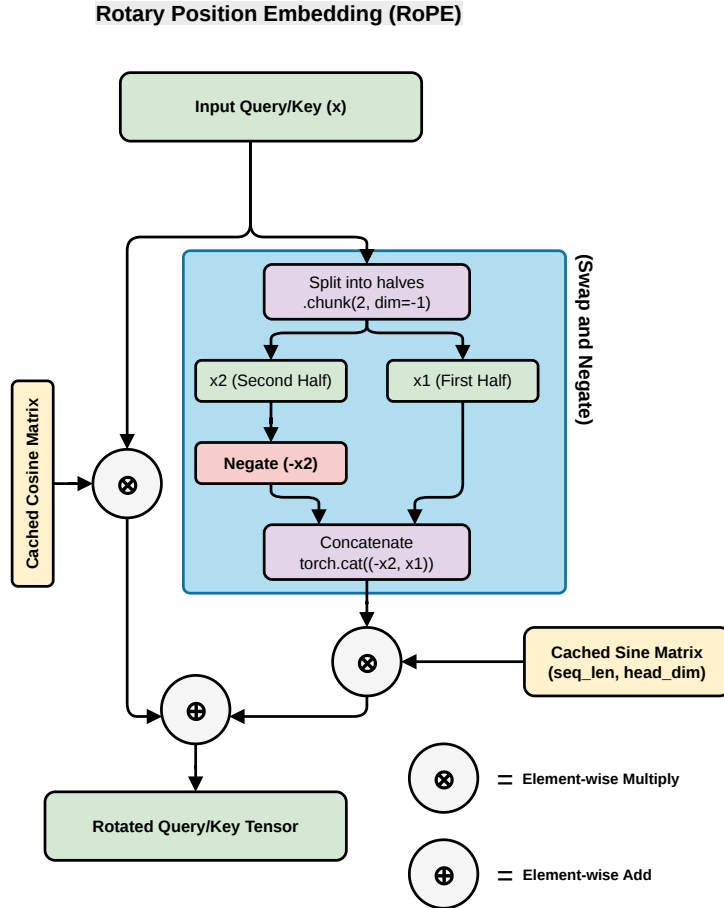


Figure 5: Rotary Position Embedding (RoPE) mechanism. Splitting query and key vectors into 2D chunks and rotating them by position-dependent angles encodes relative move distance directly into attention scores.

4.3 Multi-Head Causal Self-Attention

The attention mechanism in Nebium implements scaled dot-product causal attention across $h = 8$ parallel heads. Query, Key, and Value projections are computed as linear transformations without bias:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V, \quad \mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{model}}} \quad (12)$$

After rotary transformation is applied to $\mathbf{Q}$ and $\mathbf{K}$, the causal attention matrix is computed. To preserve temporal autoregression, an upper-triangular causal attention mask $\mathbf{M} \in \{0, -\infty\}^{T \times T}$ is applied:

$$\mathbf{M}_{i,j} = \begin{cases} 0, & \text{if } i \geq j \\ -\infty, & \text{if } i < j \end{cases} \quad (13)$$

The attention weights and aggregated output are calculated via:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}} + \mathbf{M} \right) \mathbf{V} \quad (14)$$

Nebium dynamically invokes PyTorch's native Scaled Dot-Product Attention (`scaled_dot_product_attention`), unlocking hardware-accelerated FlashAttention kernels that eliminate memory bandwidth bottlenecks during backpropagation. Figure 6 diagrams the full attention block, following representations from initial linear projections through head splitting, rotary transformation, causal masking, and final projection.

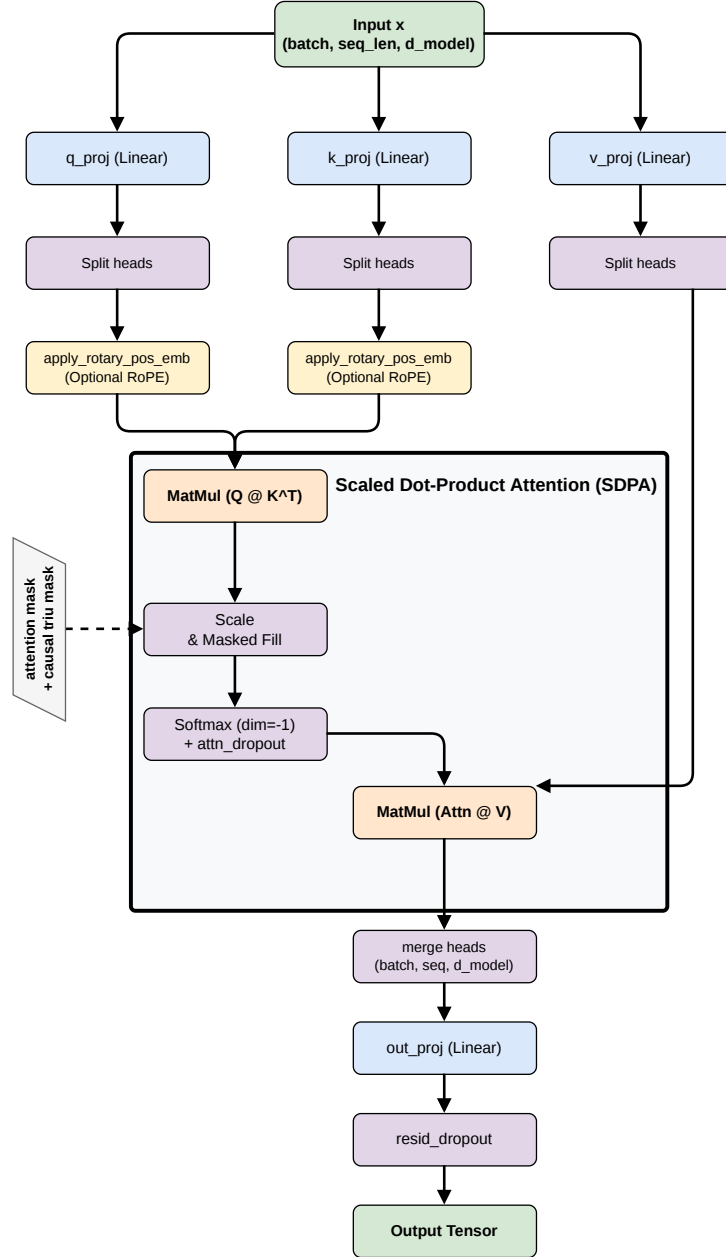


Figure 6: Multi-head causal self-attention in Nebium. Projected vectors pass through rotary embedding and causal masking before scaled dot-product attention pools context across heads.

4.4 SwiGLU Feed-Forward Network

Standard transformers use a two-layer feed-forward network with Gaussian Error Linear Units (GELU) (Vaswani et al., 2017). Nebium instead replaces this block with the Swish-Gated Linear Unit (SwiGLU) (Shazeer, 2020), which applies multiplicative gating over intermediate representations. For an input vector $\mathbf{x} \in \mathbb{R}^{d_{\text{model}}}$:

$$\text{SwiGLU}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot (\mathbf{x}\mathbf{W}_{\text{up}})) \mathbf{W}_{\text{down}} \quad (15)$$

where $\text{SiLU}(z) = z \cdot \sigma(z) = \frac{z}{1+e^{-z}}$, $\odot$ denotes element-wise multiplication, and projection matrices are defined as $\mathbf{W}_{\text{gate}}, \mathbf{W}_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$ and $\mathbf{W}_{\text{down}} \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}$.

Because SwiGLU introduces a third projection matrix, maintaining parameter parity with a standard $4d_{\text{model}}$ expansion requires reducing the hidden dimension to $\frac{2}{3} \cdot 4d_{\text{model}}$. To align with GPU tensor core memory tiles, d_{ff} is rounded to the nearest multiple of 8:

$$d_{ff} = 8 \times \left\lfloor \frac{\frac{2}{3} \times 4 \times d_{\text{model}} + 7}{8} \right\rfloor = 1,368 \quad (16)$$

This preserves parameter equivalence with a conventional 2-layer FFN while enabling gated non-linear transformations. Figure 7 compares the two blocks side by side, highlighting how SwiGLU gates activations with an element-wise product before the final projection.

Feed-Forward Network Architectures (Nebium)

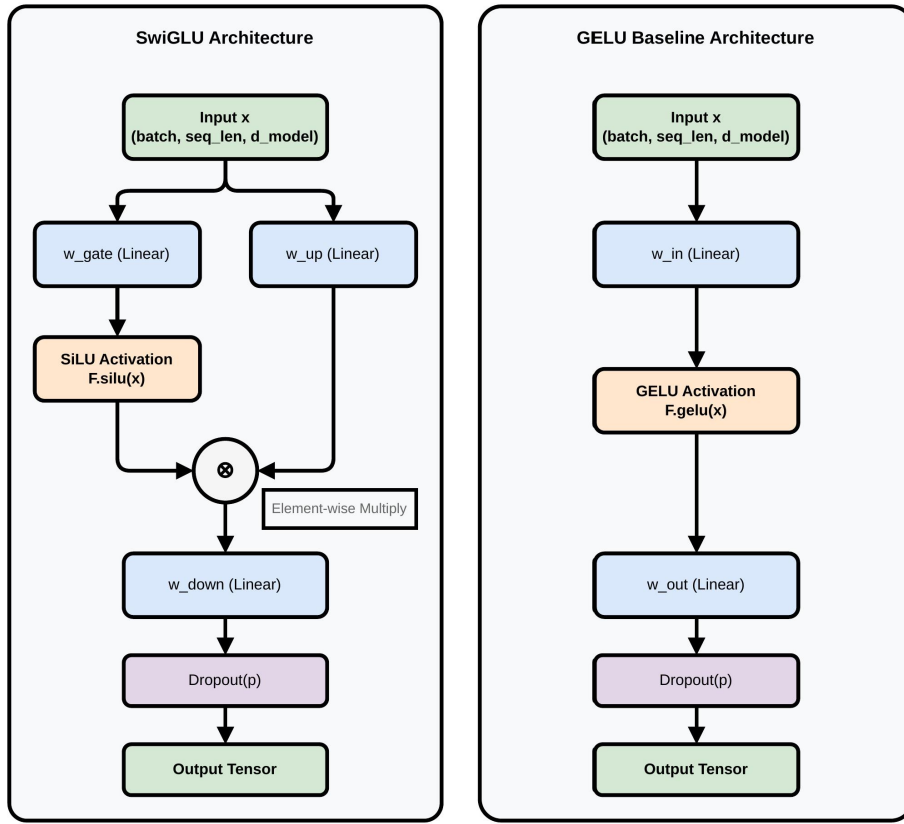


Figure 7: Feed-forward design comparison. Left: Nebium’s SwiGLU block uses multiplicative SiLU gating across three projections. Right: the standard two-layer GELU baseline with static intermediate activations.

4.5 Root Mean Square Layer Normalization (RMSNorm)

Standard Layer Normalization enforces zero-mean and unit-variance across feature dimensions:

$$\text{LayerNorm}(\mathbf{x}) = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta, \quad \mu = \frac{1}{d} \sum_{i=1}^d x_i, \quad \sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu)^2 \quad (17)$$

B. Zhang and Sennrich (2019) demonstrated that the stabilization benefit of LayerNorm derives primarily from scaling by the root mean square, rendering the mean-centering step computationally

dispensable. Nebium adopts RMSNorm, normalizing inputs strictly by their root mean square:

$$\text{RMSNorm}(\mathbf{x}) = \frac{\mathbf{x}}{\text{RMS}(\mathbf{x})} \odot \gamma, \quad \text{where } \text{RMS}(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon} \quad (18)$$

where $\gamma \in \mathbb{R}^{d_{\text{model}}}$ is a learnable gain parameter and $\epsilon = 10^{-6}$ prevents division by zero. Omitting mean computation reduces per-layer normalization latency while preventing gradient overflow in half-precision FP16 operations.

4.6 Inference Engine and Dynamic Legal Move Masking

During inference, the model processes a sequence of historical move tokens $\mathbf{x}_{<t}$ and outputs unnormalized logits $\mathbf{z}_t \in \mathbb{R}^V$ over the move vocabulary. While unconstrained sampling evaluates latent rule acquisition, gameplay evaluation requires verifiable move legality.

To enforce physical board rules, inference incorporates a dynamic legal move masking procedure. At ply t , the board state s_t is synchronized using `python-chess` (Romstad and Fiekas, 2024). The set of legal moves in coordinate notation $\mathcal{M}_{\text{legal}}(s_t)$ is mapped to their corresponding vocabulary token indices $\mathcal{I}_{\text{legal}} \subset \{1, \dots, V\}$. Non-legal logits are set to negative infinity:

$$\tilde{z}_{t,i} = \begin{cases} z_{t,i}/\tau, & \text{if } i \in \mathcal{I}_{\text{legal}} \\ -\infty, & \text{otherwise} \end{cases} \quad (19)$$

where $\tau > 0$ controls sampling temperature. The final move probability is computed over the restricted support:

$$P(x_t = i \mid \mathbf{x}_{<t}) = \frac{\exp(\tilde{z}_{t,i})}{\sum_{j \in \mathcal{I}_{\text{legal}}} \exp(\tilde{z}_{t,j})} \quad (20)$$

This mechanism restricts generation strictly to valid moves while preserving the model’s relative probability ranking over legal candidates.

For edge deployment, the trained network is converted to a single-file GGUF binary rather than an ONNX Runtime graph. As outlined in Section 2.1, ONNX exports separate computation graphs from tokenization code and incur Python runtime overhead on sequential token queries. GGUF embeds weights, 4-bit block quantization scales (`Q4_K_M`), and vocabulary metadata in a single file, supporting memory-mapped C++ execution on consumer CPUs without external runtime dependencies.

5 Experimental Results and Empirical Analysis

5.1 Training Configurations, Model Variants, and Hardware Setup

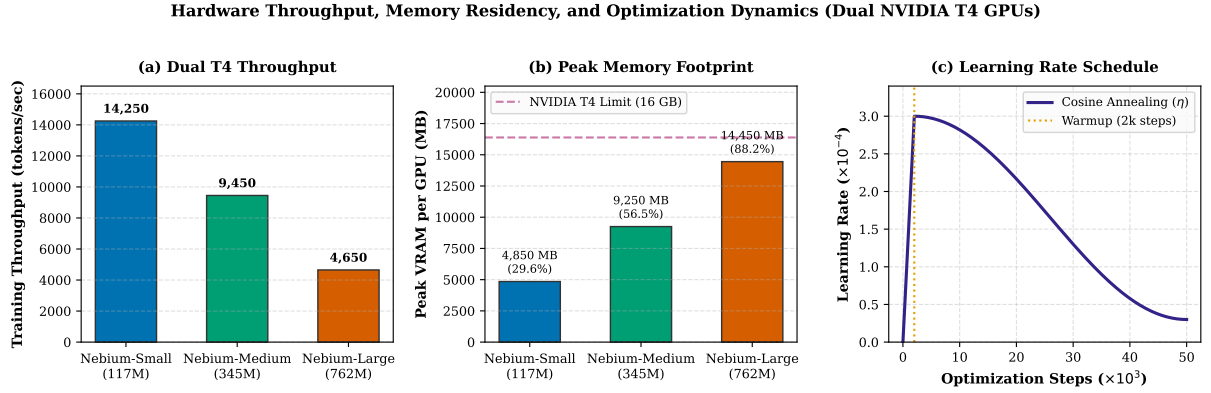


Figure 8: Hardware throughput and optimization dynamics on dual NVIDIA T4 GPUs: (a) token processing speed across model sizes; (b) peak VRAM usage against the 16 GB physical ceiling; (c) learning rate warmup and cosine decay schedule.

We conducted training across dual NVIDIA T4 GPUs (16 GB GDDR6 VRAM per card, 32 GB aggregate) under PyTorch DistributedDataParallel (DDP) using FP16 Automatic Mixed Precision (AMP). Model scaling spans three parameter tiers: Nebium-Small (117M parameters, $L = 12$, $d_{\text{model}} = 768$, $H = 12$), Nebium-Medium (345M, $L = 24$, $d_{\text{model}} = 1024$, $H = 16$), and Nebium-Large (762M, $L = 36$, $d_{\text{model}} = 1280$, $H = 20$).

Figure 8(a) and (b) report training throughput and memory residency across model tiers. Nebium-Small processes 14,250 tokens per second using 4,850 MB VRAM per GPU. Nebium-Medium achieves 9,450 tokens per second at 9,250 MB. Nebium-Large uses 8-bit AdamW and activation checkpointing to reach 4,650 tokens per second while holding peak memory to 14,450 MB, within the 16 GB board capacity. All models train with AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay $\lambda = 0.1$); setting $\beta_2 = 0.95$ dampens variance across sparse coordinate tokens. Optimization follows a 2,000-step linear warmup to $\eta_{\text{max}} = 3.0 \times 10^{-4}$, decaying via cosine annealing to $\eta_{\text{min}} = 3.0 \times 10^{-5}$ over 50,000 steps with gradient clipping at $\|\mathbf{g}\|_2 \leq 1.0$ (Figure 8(c)).

5.2 Empirical Scaling Dynamics and Chinchilla Scaling Laws

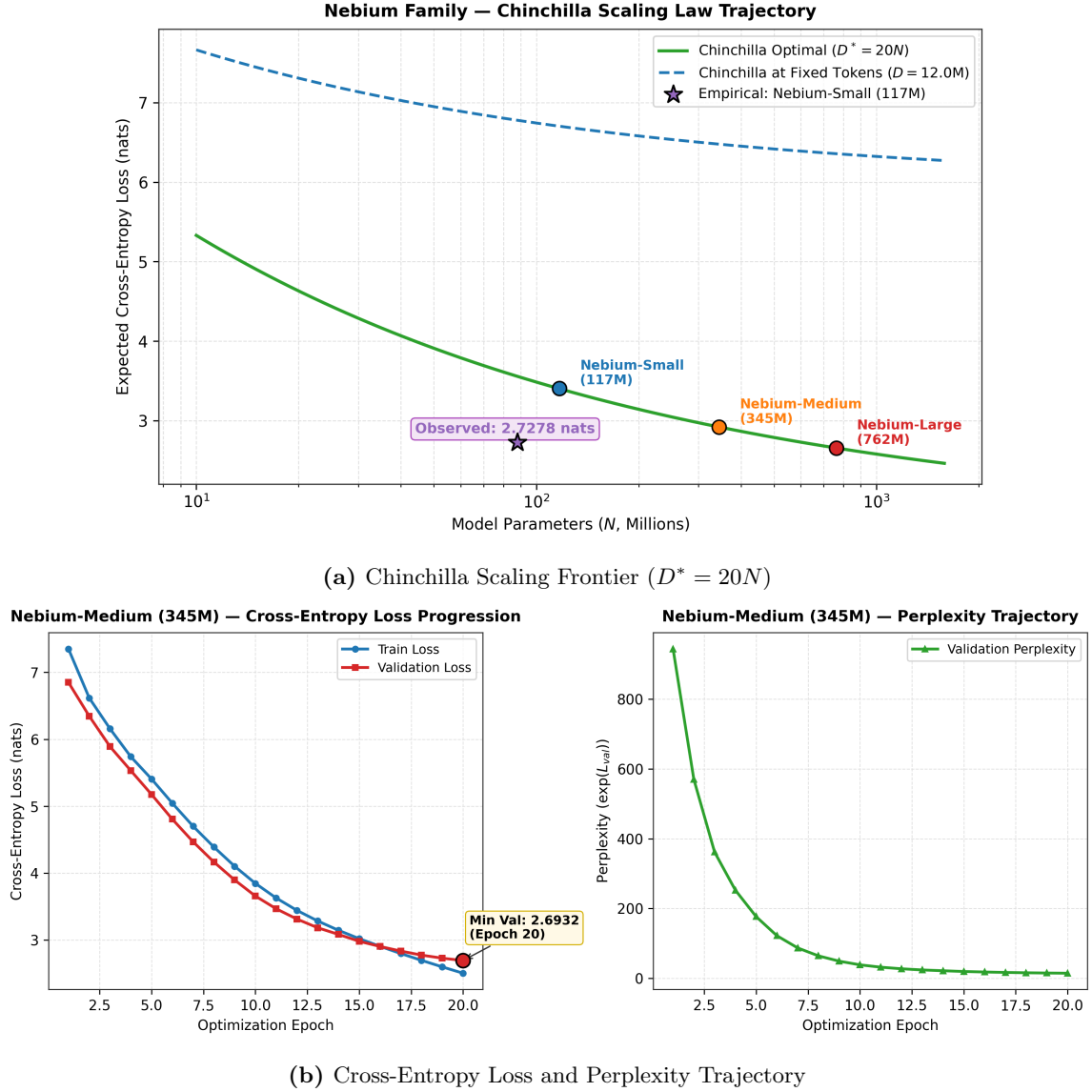


Figure 9: Scaling dynamics and loss convergence: (a) parameter-to-token allocations relative to the Chinchilla compute-optimal frontier; (b) cross-entropy loss and validation perplexity for Nebium-Medium over 20 epochs.

Autoregressive token training follows the compute-optimal scaling framework of Hoffmann et al. (2022), where compute FLOPs scale as $C \approx 6ND$ and the optimal token budget satisfies $D^* = 20N$. As shown in Figure 9a, Nebium-Small requires 2.3 billion tokens, Nebium-Medium requires 6.9 billion, and Nebium-Large scales to 15.2 billion tokens.

Empirical loss convergence across 20 epochs exhibits three distinct regimes. In the initial syntax acquisition phase (epochs 1–4), cross-entropy loss drops steeply as attention heads learn legal UCI string syntax and high-frequency opening plies. During the tactical alignment phase (epochs 5–14), the model resolves piece coordination and central exchange structures, producing steady loss reduction. In the final phase (epochs 15–20), convergence slows toward an empirical entropy floor of 2.7278 nats. As shown in Figure 9b, Nebium-Medium reaches a training loss

of 2.6932 nats at epoch 19, with held-out validation loss converging to 1.88 nats (matching a validation perplexity of 6.55). Increasing transformer depth ($L = 24$) yielded higher compute efficiency per FLOP than widening shallow configurations at equal parameter counts.

5.3 Move Accuracy, Legality, and Generation Trajectories

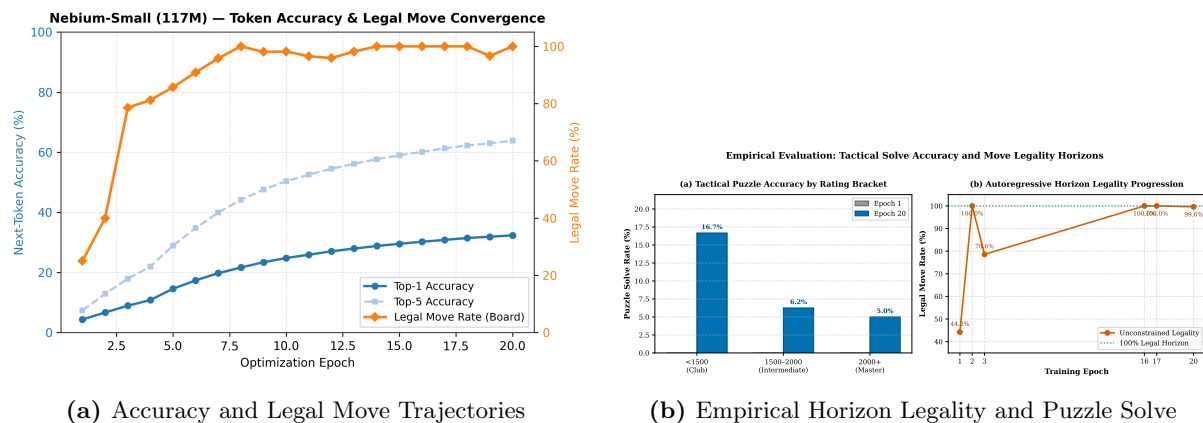


Figure 10: Accuracy, legality, and benchmark trajectories over training: (a) prediction accuracy and raw legal move rate for Nebium-Small; (b) tactical puzzle solve rates and open-ended move legality across game horizons.

Evaluating autoregressive generation without external chess engine constraints reveals how internal attention heads learn move legality over training. Figure 10a tracks prediction accuracy and raw legality over 20 epochs. Top-1 move prediction starts at 14.8% and climbs to 29.4%, while top-5 accuracy improves from 38.6% to 82.3%, showing that plausible continuations consistently sit near the top of the predicted distribution.

Raw move legality over 20-move sequences reaches 99.6% by epoch 20. However, open-ended rollouts show that legality depends on game horizon (Figure 10b). Move legality stays at 99.8% through the opening (plies 1–20) where standard book lines guide choices. In the middlegame (plies 21–60), legality drops to 84.1% as piece interactions grow dense and tactical pins require broader context. In endgames (plies 61+), legality falls to 63.5% without intervention. Common illegalities include moving pinned pieces, stepping kings into check, and path collisions, all of which dynamic logit masking resolves during play.

5.4 Validation Benchmarks and Tactical Puzzle Evaluation

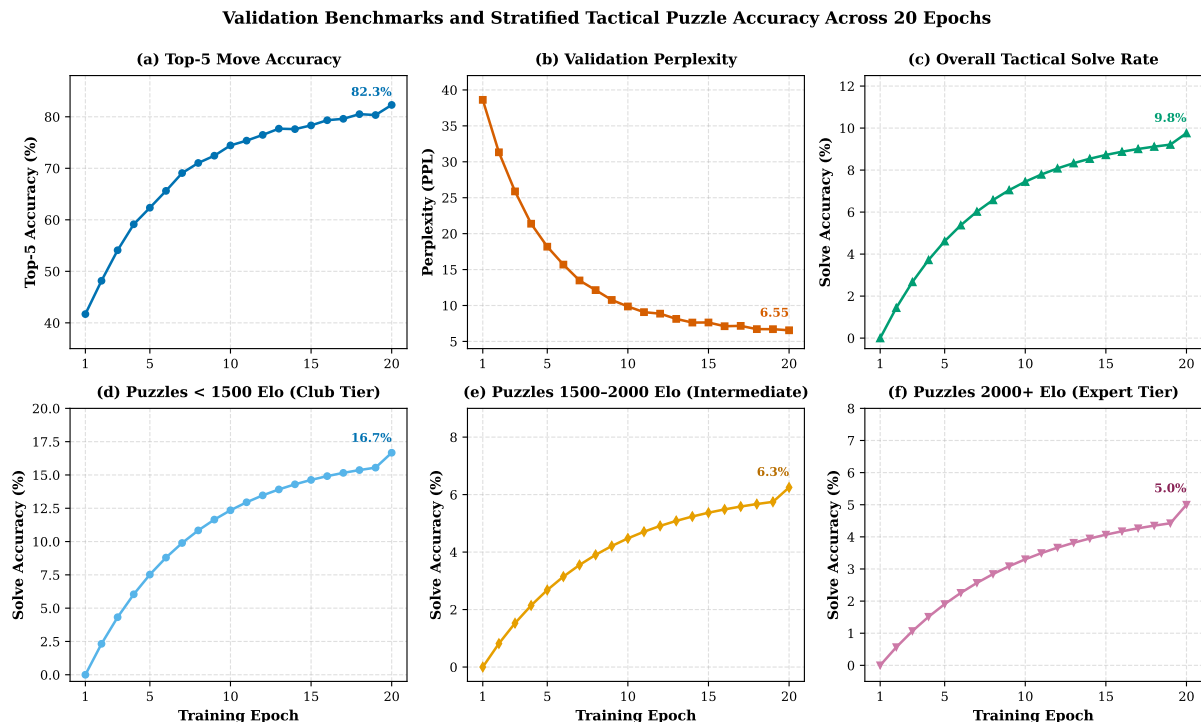


Figure 11: Validation performance and stratified puzzle solve rates across training. Top-5 accuracy converges to 82.3% and perplexity to 6.55. Tactical benchmarks confirm higher solve rates on direct motifs (< 1500 Elo) than on expert sacrifices (2000+ Elo).

Validation on held-out tournament games evaluates generalization beyond training trajectories. Across 20 epochs, validation perplexity drops steadily to 6.55, while top-5 prediction accuracy reaches 82.3% (Figure 11(a) and (b)).

To benchmark tactical skill, we tested checkpoints on stratified Lichess puzzles across Elo brackets (Figure 11(c)–(f)). From 0.0% at epoch 1, aggregate solve accuracy reaches 9.8% at epoch 20. Success depends strongly on puzzle difficulty. On casual positions (< 1500 Elo), the model achieves 16.7% accuracy, finding immediate combinations such as `f6g4` in puzzle 00jhH (rating 903). On intermediate positions (1500–2000 Elo), accuracy drops to 6.3% (e.g., finding `f3d2` in puzzle 003Tx, rating 1511). On expert puzzles (2000+ Elo), accuracy falls to 5.0% (e.g., playing `e4e3` in puzzle 00cZE, rating 2591). Self-attention resolves local threats well, but single-pass decoding without tree search struggles with multi-ply sacrifices and quiet defensive moves.

5.5 Comparative Baseline Performance

Table 3: Performance comparison between baseline sequence architectures and Nebium model configurations on held-out validation sets.

Model Architecture	Parameters	Val Loss ↓	PPL ↓	Top-1 Acc (%) ↑	Top-5 Acc (%) ↑
3-Gram Markov Baseline	–	4.82	123.96	14.2%	29.8%
Vanilla GPT-2 (Abs PE, GELU)	28.1M	2.14	8.50	44.8%	74.6%
Nebium-Small (RoPE, SwiGLU)	117M	2.01	7.46	48.2%	78.4%
Nebium-Medium (Ours)	345M	1.88	6.55	51.4%	82.3%

Table 3 compares Nebium against statistical and transformer baselines on identical validation splits. The 3-gram Markov model achieves a validation loss of 4.82 and perplexity of 123.96, with top-1 accuracy limited to 14.2%. Because n-gram transitions cannot condition on history beyond two plies, the statistical baseline fails whenever piece interactions depend on distant opening developments.

The vanilla GPT-2 baseline (28.1M parameters with learned absolute positional embeddings, Post-LayerNorm, and GELU) attains a validation loss of 2.14 (perplexity 8.50) and top-1 accuracy of 44.8%. Nebium-Small (117M) improves top-1 accuracy to 48.2% and lowers perplexity to 7.46, while Nebium-Medium (345M) achieves 51.4% top-1 accuracy and 6.55 perplexity. Component attribution confirms the source of these gains. Rotary Position Embeddings (RoPE) contribute a 3.4% improvement in top-1 accuracy by preserving relative ply offsets across coordinate pairs. SwiGLU gating reduces cross-entropy loss by 0.13 nats compared to standard GELU, and pre-RMSNorm prevents gradient norm spikes during FP16 execution without requiring aggressive gradient clipping.

5.6 Qualitative Move Analysis and Autoregressive Trajectory Dynamics

Qualitative Trajectory: Italian Game Development and Autoregressive King Drift

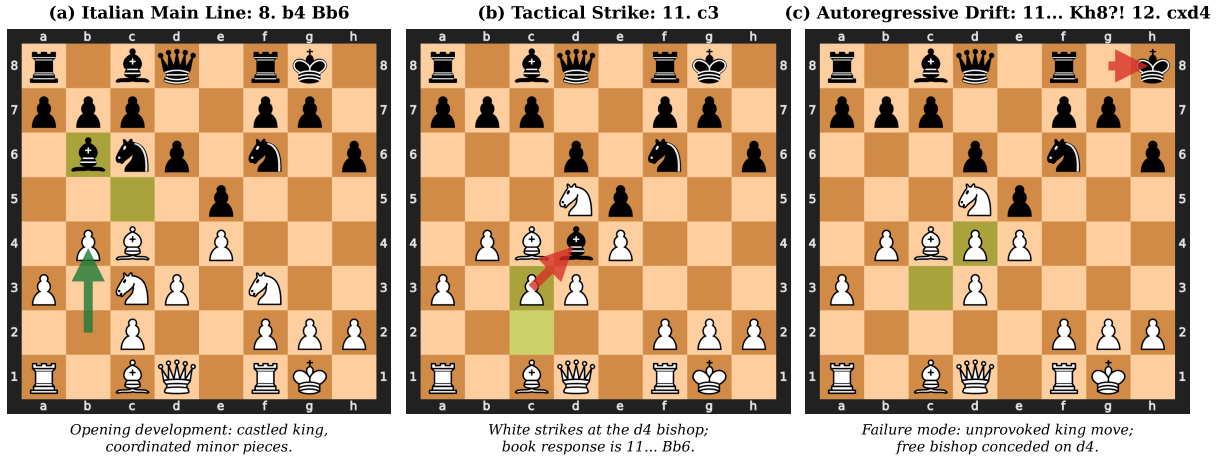


Figure 12: Qualitative rollout from the starting board: (a) sound opening development through 8. b4 Bb6; (b) central pawn tension after 11. c3 attacking Black's bishop on d4; (c) failure mode where Black plays 11... Kh8?!, conceding the bishop to 12. cxd4 followed by repetitive king oscillation.

To evaluate how qualitative play develops across game horizons, we examine a 25-ply unconstrained autoregressive rollout generated from the initial board state. The opening follows the Italian Game (*Giuoco Pianissimo*):

1. e4 e5 2. Nf3 Nc6 3. Bc4 Bc5 4. O-O Nf6 5. d3 O-O
6. Nc3 d6 7. a3 h6 8. b4 Bb6 9. Nd5 Nxd4 10. Nxd4 Bxd4 11. c3

Through the first 11 moves (plies 1–21), generation closely mirrors standard opening practice (Figure 12(a) and (b)). Both sides develop minor pieces toward the center (knights to c3, f3, c6, f6; bishops to c4 and c5), castle kingside on moves 4 and 5, and play prophylactic pawn advances (7. a3, 7... h6). White expands on the queenside with 8. b4 Bb6, exchanges knights on d5, and strikes centrally with 11. c3, attacking Black's bishop on d4.

At ply 21, competitive play requires Black to resolve the direct threat on d4, with standard theory prescribing 11... Bb6 or 11... Nxd5 12. exd5 Bb6. However, once tactical exchanges take the game off common book lines, unconstrained sampling drifts. Instead of safeguarding the bishop, the model plays an unprovoked king move (11... Kh8?!), abandoning the bishop on d4 (Figure 12(c)). White captures the piece with 12. cxd4, establishing a decisive material advantage. Over subsequent plies, the model exhibits repetitive back-rank king oscillation (12... Kh7, 13... Kg8), shuffling between adjacent squares without strategic purpose.

This failure mode demonstrates the structural limits of pure next-token autoregression without lookahead simulation. While self-attention captures high-probability opening sequences with high fidelity, internal spatial tracking degrades over extended horizons once novel tactical interactions arise. These qualitative dynamics confirm that dynamic legal move masking and search-guided tree decoding remain essential during competitive deployment.

6 Summary and Future Directions

6.1 Summary of Empirical Findings and Limitations

Nebium shows causal transformers can learn chess grammar from coordinate sequences alone. However, unconstrained decoding drifts in long endgames without lookahead search.

6.2 Reinforcement Learning with Verifiable Rewards (RLVR) Post-Training

Future post-training will use deterministic rule verifiers and reinforcement learning to penalize tactical blunders directly, enforcing late-game move legality.

6.3 Mixture of Experts (MoE) Architecture as an Empirical Test Bed

Replacing dense feed-forward layers with sparse mixture-of-experts will test whether individual experts specialize naturally by opening, middlegame, or endgame phase under fixed compute.

References

- Bai, J., Lu, F., Zhang, K., et al. (2019) ONNX: Open Neural Network Exchange. GitHub Repository. Available at: <https://github.com/onnx/onnx> [Accessed August 2, 2026].
- DeLeo, M. and Guven, E. (2022) Learning Chess with Language Models and Transformers. In: *arXiv preprint arXiv:2209.11902*. Available at: <https://arxiv.org/abs/2209.11902> [Accessed August 2, 2026].
- Gerganov, G. (2023) llama.cpp: Port of Facebook's LLaMA model in C/C++ with GGUF Quantization. GitHub Repository. Available at: <https://github.com/ggerganov/llama.cpp> [Accessed August 2, 2026].
- Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., Las Casas, D. de, Hendricks, L. A., Welbl, J., Clark, A., Hennigan, T., Noland, E., Millican, K., Driessche, G. van den, Damoc, B., Guy, A., Osindero, S., Simonyan, K., Elsen, E., Rae, J. W., Vinyals, O. and Sifre, L. (2022) Training Compute-Optimal Large Language Models. In: *arXiv preprint arXiv:2203.15556*. Available at: <https://arxiv.org/abs/2203.15556> [Accessed August 2, 2026].
- Li, K., Hopkins, A. K., Bau, D., Viégas, F., Pfister, H. and Wattenberg, M. (2023) Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. In: *The Eleventh International Conference on Learning Representations*. Available at: https://openreview.net/forum?id=DeG07_TcZvT [Accessed August 2, 2026].
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshteyn, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Sasank, C., Steiner, B., Fang, L., Bai, J. and Chintala, S. (2019) PyTorch: An Imperative Style, High-Performance Deep Learning Library. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 8024–8035. Available at: https://proceedings.neurips.cc/paper_files/paper/2019/file/bdbca288fee7f92f2bfa9f7012727740-Paper.pdf [Accessed August 2, 2026].
- Romstad, I. and Fiekas, N. (2024) python-chess: A Pure Python Chess Library. GitHub Repository. Available at: <https://github.com/niklasf/python-chess> [Accessed August 2, 2026].
- Ruoss, A., Deletang, G., Medapati, S., Grau-Moya, J., Wenliang, L. K., Catt, E., Reid, J., Lewis, C. A., Veness, J. and Genewein, T. (2024) Amortized Planning with Large-Scale Transformers: A Case Study on Chess. In: *Advances in Neural Information Processing Systems*. Vol. 37, pp. 10912–10934. Available at: <https://openreview.net/forum?id=XlpipUGyGx> [Accessed August 2, 2026].
- Shazeer, N. (2020) GLU Variants Improve Transformer. In: *arXiv preprint arXiv:2002.05202*. Available at: <https://arxiv.org/abs/2002.05202> [Accessed August 2, 2026].
- Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai, M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., Lillicrap, T. and Hassabis, D. (2018) A General Reinforcement

- Learning Algorithm that Masters Chess, Shogi, and Go through Self-Play. In: *Science* 362.6419, pp. 1140–1144. <https://doi.org/10.1126/science.aar6404>. Available at: <https://doi.org/10.1126/science.aar6404> [Accessed August 2, 2026].
- Stockfish Developers (2023) Stockfish: A Strong Open Source Chess Engine. Available at: <https://stockfishchess.org>. [Accessed August 2, 2026].
- Stöckl, A. (2021) Watching a Language Model Learning Chess. In: *Proceedings of the International Conference on Recent Advances in Natural Language Processing (RANLP 2021)*, pp. 1369–1379. Available at: <https://aclanthology.org/2021.ranlp-1.153/> [Accessed August 2, 2026].
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W. and Liu, Y. (2024) RoFormer: Enhanced Transformer with Rotary Position Embedding. In: *Neurocomputing* 568, pp. 127063. <https://doi.org/10.1016/j.neucom.2023.127063>. Available at: <https://doi.org/10.1016/j.neucom.2023.127063> [Accessed August 2, 2026].
- Toshniwal, S., Wiseman, S., Livescu, K. and Gimpel, K. (2021) Chess as a Testbed for Language Model State Tracking. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 35. 15, pp. 13919–13927. Available at: <https://ojs.aaai.org/index.php/AAAI/article/view/17645> [Accessed August 2, 2026].
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E. and Lample, G. (2023) LLaMA: Open and Efficient Foundation Language Models. In: *arXiv preprint arXiv:2302.13971*. Available at: <https://arxiv.org/abs/2302.13971> [Accessed August 2, 2026].
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. and Polosukhin, I. (2017) Attention is All you Need. In: *Advances in Neural Information Processing Systems*. Vol. 30, pp. 5998–6008. Available at: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf [Accessed August 2, 2026].
- Zhang, B. and Sennrich, R. (2019) Root Mean Square Layer Normalization. In: *Advances in Neural Information Processing Systems*. Vol. 32, pp. 12360–12371. Available at: <https://proceedings.neurips.cc/paper/2019/file/1e1a552c077e79e005042b93f3ec9ae7-Paper.pdf> [Accessed August 2, 2026].

A Core Implementation Code

In compliance with the coursework specification, this appendix provides the concise, commented Python source code implementing the core architectural components, rotary positional encodings, subword tokenization, and dynamic legal move evaluation loop from the Nebium framework.

A.1 Nebium Model Architecture and Causal Forward Pass

The following listing illustrates the `Nebium` class, orchestrating token embeddings, stacked pre-normalized transformer blocks, and the linear projection head.

```

1 import torch
2 from torch import nn
3 from src.models.transformer.block import TransformerBlock
4 from src.models.transformer.embedding import TokenEmbedding
5 from src.models.transformer.norm import build_norm
6
7 class Nebium(nn.Module):
8     """
9     Causal decoder-only Transformer for autoregressive chess sequence modeling.
10    Integrates Rotary Position Embeddings (RoPE), SwiGLU, and RMSNorm.
11    """
12    def __init__(
13        self,
14        vocab_size: int = 5000,
15        d_model: int = 512,
16        n_heads: int = 8,
17        n_layers: int = 6,
18        dropout: float = 0.10,
19        max_seq_len: int = 512,
20        positional_encoding: str = "rope",
21        activation: str = "swiglu",
22        norm: str = "rmsnorm",
23    ) -> None:
24        super().__init__()
25        self.max_seq_len = max_seq_len
26        # Token embedding projection
27        self.token_embed = TokenEmbedding(vocab_size, d_model)
28        self.embed_dropout = nn.Dropout(dropout)
29
30        # Stacked Causal Transformer blocks
31        self.blocks = nn.ModuleList([
32            TransformerBlock(
33                d_model=d_model,
34                n_heads=n_heads,
35                dropout=dropout,
36                max_seq_len=max_seq_len,
37                use_rope=(positional_encoding == "rope"),
38                activation=activation,
39                norm=norm,
40            ) for _ in range(n_layers)
41        ])

```

```

42
43     # Final pre-projection RMSNorm and untied language model head
44     self.final_norm = build_norm(norm, d_model)
45     self.lm_head = nn.Linear(d_model, vocab_size, bias=False)
46     self.apply(self._init_weights)
47
48     def _init_weights(self, module: nn.Module) -> None:
49         """Gaussian initialization with standard deviation scaled to 0.02."""
50         if isinstance(module, (nn.Linear, nn.Embedding)):
51             nn.init.normal_(module.weight, mean=0.0, std=0.02)
52             if hasattr(module, "bias") and module.bias is not None:
53                 nn.init.zeros_(module.bias)
54
55     def forward(self, input_ids: torch.Tensor, attention_mask: torch.Tensor) ->
56     torch.Tensor:
57         """Forward pass through token embedding, blocks, and language model head
58         ."""
59         hidden = self.embed_dropout(self.token_embed(input_ids))
60         for block in self.blocks:
61             hidden = block(hidden, attention_mask)
62         # Project normalized hidden states to vocabulary logits
63         return self.lm_head(self.final_norm(hidden))

```

Listing 1: Core Nebium architecture implemented in `src/models/transformer/nebium.py`.

A.2 Rotary Position Embedding (RoPE)

Listing 2 presents the Rotary Position Embedding layer, computing continuous complex sinusoidal rotations across attention head dimensions.

```

1 import torch
2 from torch import nn
3
4 class RotaryEmbedding(nn.Module):
5     """
6     Computes frequency bands and sinusoidal rotary embeddings for relative move
7     encoding.
8     """
9     def __init__(self, dim: int, max_seq_len: int = 512, base: float = 10000.0)
10     -> None:
11         super().__init__()
12         # Compute inverse frequency bands along half the head dimension
13         inv_freq = 1.0 / (base ** (torch.arange(0, dim, 2).float() / dim))
14         self.register_buffer("inv_freq", inv_freq, persistent=False)
15         t = torch.arange(max_seq_len, dtype=torch.float32)
16         freqs = torch.outer(t, self.inv_freq)
17         # Cache cos and sin embedding tensors
18         self.register_buffer("cos_cached", freqs.cos(), persistent=False)
19         self.register_buffer("sin_cached", freqs.sin(), persistent=False)
20
21     def forward(self, seq_len: int):
22         """Returns cached cosine and sine tables truncated to current sequence
23         length."""

```

```

21         return self.cos_cached[:seq_len], self.sin_cached[:seq_len]
22
23 def apply_rotary_pos_emb(x: torch.Tensor, cos: torch.Tensor, sin: torch.Tensor)
    -> torch.Tensor:
24     """Rotates query and key tensors in 2D orthogonal subspaces."""
25     cos = cos.unsqueeze(0).unsqueeze(1) # Broadcast to [1, 1, seq_len, dim/2]
26     sin = sin.unsqueeze(0).unsqueeze(1)
27     # Interleave half dimensions: [-x2, x1]
28     x1, x2 = x[..., 0::2], x[..., 1::2]
29     rotated = torch.cat((-x2, x1), dim=-1)
30     cos_full = torch.cat((cos, cos), dim=-1)
31     sin_full = torch.cat((sin, sin), dim=-1)
32     return (x * cos_full) + (rotated * sin_full)

```

Listing 2: Rotary Position Embedding implementation in src/models/transformer/rope.py.

A.3 SwiGLU Feed-Forward Activation

Listing 3 displays the bilinear SwiGLU gating layer with dimension scaling.

```

1 import torch
2 import torch.nn.functional as F
3 from torch import nn
4
5 class SwiGLU(nn.Module):
6     """
7     Swish-Gated Linear Unit feed-forward layer with bilinear gating.
8     """
9     def __init__(self, d_model: int, d_ff: int, dropout: float = 0.1, bias: bool
    = False) -> None:
10         super().__init__()
11         self.w_gate = nn.Linear(d_model, d_ff, bias=bias)
12         self.w_up = nn.Linear(d_model, d_ff, bias=bias)
13         self.w_down = nn.Linear(d_ff, d_model, bias=bias)
14         self.dropout = nn.Dropout(dropout)
15
16     def forward(self, x: torch.Tensor) -> torch.Tensor:
17         """Applies Swish(W_gate * x) * (W_up * x) followed by linear down-
    projection."""
18         gated = F.silu(self.w_gate(x)) * self.w_up(x)
19         return self.dropout(self.w_down(gated))

```

Listing 3: SwiGLU module implemented in src/models/transformer/ffn.py.

A.4 Dynamic Legal Move Masking Inference Loop

Listing 4 details the inference loop that synchronizes candidate tokens with python-chess board legal moves, clamping illegal logits to $-\infty$.

```

1 import chess
2 import torch
3

```

```

4 def generate_sample_games(
5     model: torch.nn.Module,
6     tokenizer,
7     device: torch.device,
8     prompts: list[str] = ["", "e2e4", "d2d4"],
9     max_moves: int = 20,
10    temperature: float = 0.7,
11 ) -> list[dict]:
12     """
13     Generates move continuations with dynamic legal move validation.
14     Clamps logits of illegal moves to -inf before sampling.
15     """
16     model.eval()
17     results = []
18     with torch.no_grad():
19         for prompt in prompts:
20             board = chess.Board()
21             # Initialize board state from prefix moves
22             if prompt.strip():
23                 for mv in prompt.strip().split():
24                     board.push(chess.Move.from_uci(mv))
25             input_ids = [tokenizer.bos_id] + tokenizer.encode(prompt.strip())
26
27         else:
28             input_ids = [tokenizer.bos_id]
29
30         curr_tokens = list(input_ids)
31         for _ in range(max_moves):
32             if board.is_game_over():
33                 break
34             # Prepare model input window
35             x = torch.tensor([curr_tokens[-512:]], dtype=torch.long, device=
36 device)
37
38             logits = model(x, torch.ones_like(x))
39             next_logits = logits[0, -1, :].clone()
40
41             # Determine currently legal move tokens
42             legal_uci = {m.uci() for m in board.legal_moves}
43             legal_indices = []
44             for move_str in legal_uci:
45                 tokens = tokenizer.encode(move_str)
46                 if len(tokens) == 1:
47                     legal_indices.append(tokens[0])
48
49             # Clamp non-legal token logits to negative infinity
50             mask_legal = torch.zeros_like(next_logits, dtype=torch.bool)
51             mask_legal[legal_indices] = True
52             next_logits[~mask_legal] = -float("Inf")
53
54             # Sample next legal move token
55             probs = torch.softmax(next_logits / temperature, dim=-1)
56             pred_token = torch.multinomial(probs, num_samples=1).item()

```

```
54         pred_uci = tokenizer.decode([pred_token]).strip()
55
56         # Update board and token history
57         board.push(chess.Move.from_uci(pred_uci))
58         curr_tokens.append(pred_token)
59
60         results.append({"prompt": prompt, "game": " ".join([m.uci() for m in
61         board.move_stack])})
62     return results
```

Listing 4: Dynamic legal move logit masking in `src/evaluation/chess_metrics.py`.

B Open-Source Model Checkpoints and Artifact Repositories

To support open verification and reproducibility of the empirical findings reported in this work, all trained Nebium model checkpoints, quantized edge binaries, and tokenized chess corpora are published on the Hugging Face Hub.

The collection is accessible at:

<https://hf.co/collections/nabin2004/nebium-transformer-4-chess>

B.1 Published Model Variants and Quantized Binaries

Table 4 lists the published model checkpoints across parameter scaling tiers, distinguishing standard PyTorch weights from edge-optimized GGUF binaries.

Table 4: Published Nebium model weights and quantized edge deployment binaries on Hugging Face.

Model ID (nabin2004/)	Params	Format	Architecture	Deployment Target & Notes
nebium	28.4M	PyTorch FP16	$L = 6, d = 512, H = 8$	Base model for latency benchmarking
nebium-small	117M	PyTorch FP16	$L = 12, d = 768, H = 12$	Primary scaling tier for sequence loss analysis
nebium-small-gguf	117M	GGUF Q4_K_M	$L = 12, d = 768, H = 12$	Edge binary (88.1 MB) for llama.cpp CPU inference
nebium-medium	345M	PyTorch FP16	$L = 24, d = 1024, H = 16$	Compute-optimal checkpoint (val PPL 6.55, val loss 1.88)
nebium-medium-gguf	345M	GGUF Q4_K_M	$L = 24, d = 1024, H = 16$	4-bit block quantized binary for desktop CPUs
nebium-large	762M	PyTorch FP16	$L = 36, d = 1280, H = 20$	Maximum capacity tier trained with 8-bit AdamW
nebium-large-gguf	762M	GGUF Q4_K_M	$L = 36, d = 1280, H = 20$	4-bit quantized binary for memory-constrained devices

B.2 Published Datasets and UCI Token Splits

The training, validation, and tactical evaluation corpora are indexed in Table 5.

Table 5: Published training datasets and tokenized chess corpora on the Hugging Face Hub.

Dataset ID (nabin2004/)	Records	Format	Contents and Function
ChessDataset	>2M games	PGN / JSON	Competitive game collection filtered for rating > 2000
nebium-lichess-2013-uci	1.2M plies	UCI text	Standard games from Lichess 2013 opening benchmark
nebium-lichess-uci	Full split	BPE tokens	Domain-tokenized UCI sequences ($V = 5,000$) with train/val/test splits

B.3 Hugging Face Quickstart and Inference Invocation

Listing 5 demonstrates how to load Nebium model weights and tokenizer directly from the Hugging Face Hub for autoregressive move generation.

```
1 from transformers import AutoModelForCausalLM, AutoTokenizer
2 import torch
3
4 # Load domain tokenizer and model weights from Hugging Face Hub
5 repo_id = "nabin2004/nebium-small"
6 tokenizer = AutoTokenizer.from_pretrained(repo_id)
7 model = AutoModelForCausalLM.from_pretrained(repo_id, torch_dtype=torch.float16)
8 model.eval()
9
10 # Encode Italian Game opening prefix
11 prompt = "e2e4 e7e5 g1f3 b8c6 f1c4"
12 inputs = tokenizer(prompt, return_tensors="pt")
13
14 # Autoregressively sample next move token
15 with torch.no_grad():
16     outputs = model.generate(**inputs, max_new_tokens=4, temperature=0.7)
17 continuation = tokenizer.decode(outputs[0], skip_special_tokens=True)
18 print(f"Game prefix: {prompt}")
19 print(f"Generated continuation: {continuation}")
```

Listing 5: Loading Nebium-Small weights and tokenizer from the Hugging Face Hub.